Build / Authorship Notes — FaceTrack CRM

Author: Eric Tsou **Window**: 48 hours from 2026-05-17 receipt + two follow-up sessions (gate UX, second LLM backend) **Tooling**: solo build. I directed Claude Code (Opus 4.7) for boilerplate (SQLModel schema scaffolding, OpenCV API recall, docstring drafts). The architectural decisions in §1, the threshold re-calibration in §6, and every debug story in §3 are mine alone.

This note covers the brief's four questions: what I personally built, what I reused, what broke, and how I debugged it. Numbers in §4–§5 are reproducible — see `scripts/benchmark.py` and `scripts/reproducibility_evidence.py`.

1. What I personally built

- **The engineering invariant behind the brief.** The brief says the system must "*track consistently across visits.*" I translated that requirement into one engineering rule the codebase enforces structurally:

  > *The scoring path must be bit-identical reproducible. Run-twice on the same image returns the same five floats; an LLM never appears in that path.*

  AI Fund owns the venture thesis (Beauty Clinic OS — I attended the breakfast briefing). What I own is the call to make *deterministic CV scoring + a Photo-Consistency Gate before the database* the structural defence against the "thin Vision-LLM wrapper" failure mode. The σ̄ comparison in §4 is the empirical evidence this call was the right one.

- **The Photo-Consistency Gate** (`src/facetrack/consistency_gate.py`). Four checks (pose / exposure / sharpness / colour). I picked ArUco 5×5 over OpenCV's CharucoBoard because a single marker is easier to print on a credit-card-sized card a clinic can hand to a patient at intake — a CharucoBoard needs an A4 sheet that doesn't fit any reception workflow.
- **The scoring engine — what's mine vs. what's textbook.** The five OpenCV primitives are textbook (black-hat, Sobel, LoG, LAB a*, L* std). What I did:
  - **picked** one primitive per skin attribute (e.g. black-hat over ITA°; see §5 for the trade-off);
  - **defined** the raw-to-0-10 mapping (linear clamp with explicit `*_RAW_RANGE` constants, see `scoring.py:35`);
  - **calibrated** the four ranges empirically on three CC0 reference faces after observing first-run saturation (see bug #3 in §3);
  - **wrote** the polygon-masked statistic helpers (`_ratio_inside` / `_stat_inside`) so the displayed score corresponds to the anatomical polygon drawn on the intake view, not the enclosing bounding box.
- **The hybrid LLM-vs-CV split.** The `Explainer` interface deliberately accepts *scores* (a `dict[str, float]`), not images. The LLM is structurally prevented from making up numbers because it never sees the source pixels. This is enforced by the type signature, not by discipline.
- **Streamlit 6-page UX in 繁體中文** (`patients` / `intake` / `history` / `treatment` / `overview` / `settings`), the SQLModel schema (Patient → Visit → RegionScore → TreatmentNote), and the seed-trajectory generator used for deterministic test fixtures.

2. What I reused

- MediaPipe Face Landmarker (Tasks API) — pre-trained model, vendored as a 3.6 MB `.task` file.
- OpenCV primitives: CLAHE, ArUco, morphology, Sobel, Laplacian. Cited above as primitives, not contributions.

- Streamlit / Plotly / SQLModel — standard scaffolding.
- `thispersondoesnotexist.com` for three CC0 test faces used for pipeline calibration. No real-person images in the repo.

3. What broke and how I debugged it

1. **`from __future__ import annotations` broke SQLModel relationships.** SQLModel introspects relationship annotations at class-definition time, and PEP 563 turns them into strings it cannot resolve. The error was a deep SQLAlchemy `_resolve_name` traceback that did not mention the future import. I lost ~15 min suspecting a missing `Mapped[...]` annotation before grepping the file for what was different about `db.py` vs. every other module that worked. *Fix*: removed the future import from `db.py` only, switched `list["Visit"]` to a forward-ref + `SQLModel.model_rebuild()` pattern, and left the gotcha in the module docstring so I don't reintroduce it.

2. **`mp.solutions.face_mesh` is gone in MediaPipe 0.10.35.** Every tutorial on the internet uses the legacy `solutions` namespace, which was dropped from the macOS arm64 wheel with no deprecation warning — just a flat `AttributeError`. *Fix*: rewrote `cv_pipeline.py` against the new Tasks API (`mediapipe.tasks.python.vision.FaceLandmarker`). The unexpected payoff was that the Tasks API returns a 4×4 facial transformation matrix, so the pose check became a direct Euler-angle extraction from a 3×3 rotation block (`consistency_gate.py:223`) instead of the landmark-triangulation heuristic I had originally sketched. The forced migration ended up being a net positive.

3. **Scoring saturated at 10/10 on the first end-to-end run.** I had seeded the four `*_RAW_RANGE` constants from textbook numbers; the actual distribution on CLAHE-normalized ROIs was 5–10× higher. *How I debugged*: instrumented `scoring.py` to print `raw_*` values on the three reference faces before the clamp, looked at the actual range (e.g. pigmentation raw came in at 0.05–0.25, not the 0.005–0.03 I had hard-coded), and set each constant to roughly the observed `[p10, p90]` interval. n=3 is small and I know it — see §6 for why I am explicitly *not* claiming a percentile distribution. The `*_RAW_RANGE` constants are public knobs precisely so a clinic with its own training distribution can re-fit them.

4. **The Bash sandbox refused to download test faces over HTTPS.** I had wrapped `urllib.request.urlopen` in an `ssl.CERT_NONE` context out of habit. The denial was correct — there was no reason to disable TLS for a public Wikimedia / TPDNE URL. *Fix*: removed the SSL bypass, re-ran with default TLS, three faces downloaded in 4 s. Worth recording because it's the kind of "habit" that ends up in production by accident.

5. **The wild-goose-chase one — opaque "模型錯誤" badge cost 30 min.** In Session 2, the live face-capture component started failing with a red "模型錯誤" badge in the iframe. I assumed the vendored `face_landmarker.task` was corrupted, re-downloaded it, swapped the mirror target, even briefly tried a different MediaPipe model name — none of it helped. The actual cause was the CDN-loaded **WASM fileset** version drifting from the JS Web SDK version that imported it, which surfaced only as a generic init failure. *Fix*: pinned the WASM fileset import to a known-good version. *Permanent fix*: built an in-iframe debug log (`index.html:178` and below — cyan-bordered scrolling box + 📋 copy-to-clipboard button) that logs every setup step with `[hh:mm:ss.ms]` and ok/warn/err colour. The next time the demo breaks in the room, I copy-paste the log instead of guessing. The principle ("if a black-box error wastes more than 15 minutes, the next debug session must come with observability built in") is the real takeaway, not the WASM version itself.

4. What I measured

Reviewers asked me on draft 1 to back up the "deterministic, real-time, clinic-ready" claims with numbers. Fair. Both tables below are reproducible from the repo:

End-to-end latency (`uv run python scripts/benchmark.py`, M4 Pro MacBook, macOS 15.6, 3 CC0 faces × 5 runs each, MediaPipe Tasks on CPU via XNNPACK; re-measured 2026-07-04 after the v2 gate + scoring changes):

Stage	mean	p50	p95
Pipeline (MediaPipe align + 512px scale-norm + ROI crop)	7.8 ms	7.7	8.1
Consistency gate (6 checks)	6.3 ms	6.3	6.6
Scoring (5 metrics × 4 ROIs, glare/shadow-masked)	7.6 ms	7.6	8.1
Total (no LLM)	**21.7 ms**	**21.6**	**22.5**

(v1 measured 18.9 ms p50 with 4 checks and no effective-mask exclusion — the two new gate checks and the per-metric exclusion masks cost ~2.7 ms, still ~70× under a 60 fps frame budget.)

The LLM call is the only network hop and is excluded — at typical Sonnet 4.6 latency it dominates the total and varies with the network, which is exactly why scoring lives outside of it.

Reproducibility under mild input perturbation (`uv run python scripts/reproducibility_evidence.py`, same face × 20 small rotation/exposure/JPEG re-encode perturbations, σ of the resulting 0–10 score). The simulated stochastic baseline adds Gaussian noise at σ=1.0 — calibrated to published Vision-LLM rating-task variance, so the qualitative gap is what matters, not the exact baseline number:

Metric	σ (deterministic CV)	σ (simulated LLM baseline)
pigmentation	0.286	0.988
erythema	0.139	0.565
wrinkle	0.091	0.488
pore	0.232	0.595
uniformity	0.119	0.792
mean $\bar{\sigma}$	**0.173**	**0.686**

~4× tighter than a representative stochastic grader. This is what "tracks consistently across visits" means in numbers.

An honesty note, because the v1 table looked "better" ($\bar{\sigma}$ 0.074): two of v1's five σ values were exactly 0.000 because the wrinkle and pore scores were **saturated at 10.0** on the reference face — a clamped score cannot vary, so that part of the v1 figure was range mis-calibration masquerading as robustness. The v2 ranges put scores mid-band where perturbation sensitivity is real and visible. The number that actually matters for longitudinal tracking moved the right way: **cross-resolution drift on the same face fell from up to 5.5 points (v1) to ≤ 1.05 points (v2)** — see Session 4 below for the mechanism.

The v2-retuned wrinkle row deserves the same scrutiny in the other direction: this reference face sits in the top tail of the FFHQ population (raw wrinkle at/above the p95-calibrated 0.62 ceiling in 57 of 80 per-ROI measurements across the 20 runs), so part of its post-retune σ (0.091) is ceiling-clamp, not pure robustness. That is the accepted cost of calibrating the range to the population p5–p95 instead of to our 5 reference faces — ~5% of real faces will clamp at 10 by construction. A mid-band reference face would make the wrinkle row comparable again; noted as a follow-up.

5. Trade-offs I made deliberately

Every choice in this pipeline has an alternative I rejected. The brief asks for engineering reasoning, so the comparisons are explicit here rather than buried in commit messages.

Decision	Chosen	Alternative	Why I rejected the alternative
Pigmentation primitive	Black-hat morphology + fixed cutoff	ITA° on CIE Lab L*	CLAHE-normalised ROIs compress the L* range so ITA° clusters collapse and the threshold becomes brittle. Black-hat reads local dark structure directly — CLAHE actually helps it.
Wrinkle primitive	Isotropic Sobel-magnitude density	Gabor filter bank (8 orientations × 3 scales)	Gabor is theoretically a better fit for oriented lines but adds ~50 ms / ROI on CPU and has 24 hyperparameters I can't hand-tune in 48 h on 3 faces.
Face landmarker	MediaPipe Tasks API	dlib 68-point / InsightFace	dlib has no transformation matrix (no clean pose check). InsightFace is more accurate but its wheel is 200+ MB and would slow Streamlit Cloud cold starts to minutes.
Scoring normalization	Linear clamp against `*_RAW_RANGE`	Per-cohort percentile (e.g. "you are worse than 70 % of comparable patients")	Percentile needs a cohort. n=3 reference faces can't support that without lying about it. Linear clamp is the honest placeholder; the constants are public so a clinic re-fits them on real data.
Frontal pose tolerance	±15°	±5° (clinical-grade) / ±30° (consumer-grade)	Measured roll on six volunteers using laptop webcams: median −7.5°, IQR ~5°. ±5° rejected nearly everyone; ±30° let through obvious tilts. ±15° catches the genuine misuse without punishing natural posture.
Database	SQLite + SQLModel	Postgres	Repo ships with a working DB; reviewer clones and runs. Migration to Postgres is a one-line `DB_URL` swap when it ever matters.
Frontend	Streamlit + custom HTML component	React + FastAPI	48 h scope. The custom component already gives me the one piece Streamlit can't — in-browser MediaPipe with real-time mesh overlay.
ROIs	Anatomical polygon masks fed into scoring	Bounding-box crops	The masked statistic helpers (`_ratio_inside`) ensure the score corresponds to the polygon drawn on screen, so the visible region and the number reported are the same set of pixels.

6. Iterations after the first 48 h window

The submission did not stop at the prototype. Two follow-up sessions hardened the gate UX and the LLM layer.

Session 2 — live face-mesh capture (replaces blind `st.camera_input`)

Custom Streamlit component (`src/facetrack/components/face_capture/`) runs MediaPipe Tasks Vision in the browser, draws the 478-point mesh + region-coloured contours on a selfie-mirrored canvas, and auto-captures only when pose / face-fill / stability all pass for `LIVE_CAPTURE_STABILITY_FRAMES = 10` consecutive frames (the constant is hard-coded in `index.html:404` for now — promoting it to a Python config injected via component args is in the cut list below).

`ConsistencyGate.evaluate()` gained a `pose_mode` literal (`frontal` / `profile_left` / `profile_right`); `_check_sharpness` now measures Laplacian variance on the face bounding box (full-frame fallback when no landmarks). Five regression tests in `test_consistency_gate.py`.

Webcam-realistic threshold re-calibration. DSLR-tuned thresholds were unreachable for laptop users. Numbers tuned on six volunteers across two MacBook generations:

Knob	Original	Reset to	Why
`POSE_TOLERANCE_DEG`	8.0	**15.0**	Median roll on laptop webcams ≈ –7.5°, IQR ≈ 5°.
`SHARPNESS_MIN_LAPLACIAN_VAR`	80.0	**30.0**	Webcam JPEG variance lands at 15–20 on full frame / 35–80 on face crop. Moved measurement to the face crop and lowered the floor; the blurry-image regression test (`GaussianBlur 51/25`) still trips.
`PROFILE_YAW_MIN_DEG`	55 → 25 → **5**	**5.0**	The side photo's use case is cheek-skin sampling, not a dramatic profile. MediaPipe loses one eye past 30° and refuses to return a transform.
JS JPEG quality	0.92	**0.95**	Sharper boundary, cheap cost.

Side note for honesty: this round did mis-tune once. The `PROFILE_YAW_MIN_DEG = 25` intermediate value silently rejected all volunteers because MediaPipe stopped returning a transform before they hit 25°. I noticed only because all four side-photo tests rejected identically. The 5° final value reflects that constraint.

Session 3 — second LLM backend + history ROI overlay

- Added `GeminiExplainer` next to `AnthropicExplainer`; factory `get_explainer()` resolves backend via `LLM_BACKEND` env var, else auto-selects Anthropic → Gemini → Mock based on which keys are present. Both real backends `try/except` the SDK error path and fall back to `MockExplainer` — the demo never hard-fails. Unblocks the recording case where I have one of the two keys but not both.
- 📋 就診歷史 got two flat toggles (Streamlit forbids nested expanders): 🔬 **ROI 訊號疊圖** (re-runs `compose_intake_view` with a metric selector) and 🧪 **各 ROI 局部影像** (four CLAHE-normalised thumbnails). Both wrapped in `@st.cache_data` keyed on `(path, mtime)` so flipping toggles doesn't re-trigger MediaPipe.
- `patient_service.py` + `score_display.py` factored out of `app.py`; soft-delete is a flag flip so longitudinal history stays intact. `tests/conftest.py` adds a temp-DB fixture; `test_patient_service.py` and `test_score_display.py` lock the contracts.

A presentation accommodation worth disclosing

For the Loom recording I wanted my own face to appear as the tracking baseline. My captures of myself in the recording-room lighting failed the gate (low Laplacian on the cheek crop — the room is fluorescent and my webcam underexposes). I had a choice:

1. Lower the gate thresholds again — but I had just re-tuned them and any further relaxation would let through genuinely bad photos.
2. Add a per-patient bypass flag in code — but the whole IP claim is that the gate path is sacred.
3. Mutate the rows in my local SQLite DB directly: flip `quality_passed=True` on my three visits, leave the gate code untouched.

I picked (3). The visits in question are `visits.id ∈ {10, 11, 12}` in the local DB only; the Streamlit Cloud build re-seeds from scratch and will not include them. I am disclosing this in the build note rather than hiding it because the alternative (changing the code to make the demo look smooth) is the worse signal.

Session 4 — Gate v2 + Scoring v2 (2026-07-04)

A systematic hardening pass over the two core features, driven by measurements on the reference faces rather than intuition. Every threshold below was calibrated against `data/test_images` first; the "before" numbers are in the calibration notes, the "after" numbers in §4 above. 71 → 92 tests.

The headline bug: scores depended on camera distance. All five metrics use fixed pixel-size kernels, but nothing fixed the pixel size of a face. Measured: the *same photo* downscaled to 0.5× moved pore from 4.94 to 10.00 and wrinkle from 2.22 to 5.69 — i.e. a patient photographed with a newer phone (or 20 cm closer) would appear to get worse skin. For a product whose one promise is cross-visit comparability, this was the highest-severity defect in the codebase. Fix (three parts, each necessary):

1. **Scale normalization** — the alignment warp now also rescales the face so the anatomical width (landmark 234 ↔ 454) is exactly 512 px before ROI extraction. One warp, no extra resample.
2. **512, not 1024** — first attempt normalized to 1024 px (the scale the v1 ranges were nominally calibrated at) and the drift persisted, because a 500 px webcam face *upscaled* to 1024 has no detail above its native sampling — interpolation fabricates smoothness, not skin. 512 is the lowest-common-denominator: every accepted photo reaches it by downscaling (or ≤ 1.28× upscale), so all inputs share the same detail ceiling.
3. **Native face-width floor (400 px) at the gate** — inputs that cannot reach 512 honestly are rejected with "臉部影像過小，請靠近 鏡頭", not scored. 400 sits under the live-capture widget's own face-fill floor (448 px), so no auto-captured frame is affected.

Residual drift after 1+2+3 was concentrated in pigmentation, and diagnosing it found a real formula bug: black-hat was the only metric computed **without a denoise step**, so its fixed cutoff (>18) was counting sensor/CLAHE noise — whose amplitude varies with the downscale factor. Adding the same 3×3 Gaussian the wrinkle metric already used dropped cross-resolution drift to ≤ 1.05 points across the reference set (from up to 5.5 pre-v2).

Gate v2 — two new checks, three fixed ones:

- **Exposure now measures the face, not the frame.** Full-frame stats rejected `test_face_3` (bright wall, correctly-exposed face) and under-reported `test_face_1` (blown-out face, mid-tone wall). Face crops get a looser near-black budget (6 % vs 2 %) because pupils / eyebrows / nostrils are legitimately near-black — measured 2.2–3.2 % on healthy faces, which the old threshold would have failed.
- **Sharpness is resolution-normalized** (face crop resized to 256 px wide before the Laplacian). The v1 threshold whiplashed 80 → 30 when the capture device changed; normalized, sharp faces measure 87–494 vs 1.9–3.7 for blurred — a 20× separation band that no longer moves with the camera.
- **Lighting uniformity (new).** Left/right face-brightness asymmetry

  > 0.25 rejects. Measured separation: evenly-lit 0.065–0.143 vs side-lit 0.31–0.52. Side light is the most common real clinic failure (window seating) and it directly biases the left-vs-right cheek comparison — no per-pixel exposure stat catches it.

- **Skin visibility (new).** Per-ROI YCrCb skin-pixel ratio < 0.35 rejects, naming the region. This ships LIMITATIONS §2's occlusion plan: real-face ROIs measure ≥ 0.50, mask fabric / sunglasses 0.00. A masked selfie previously passed every check and scored the fabric.
- **White-balance gains clamped to [0.6, 1.8]** — a glare-corrupted gray-card sample could previously recolor the whole image by an unbounded factor, corrupting erythema worse than no calibration.

Scoring v2 — measure skin, not lighting artifacts:

- Every metric now runs on an **effective mask** = ROI polygon minus specular pixels (L* > 235) and deep shadow (L* < 20), eroded 1 px to drop the artifact-to-skin transition ring. Forehead glare was being scored

as "non-uniform tone" and its rim as "edges"; near-black hair strands as melanin.

- **Calibrated pixels are now the scored pixels.** When the gray-card fires, the pipeline re-runs on the calibrated image before scoring. Previously the calibrated photo was *saved* but the *uncalibrated* ROIs were scored — so re-scoring a stored photo would not reproduce its stored score, quietly breaking the reproducibility contract the TDD §3 promises (and the erythema metric never actually received the color correction it was documented to rely on).
- **`SCORING_VERSION` (= 2) persisted per visit**, with a zero-downtime migration backfilling old rows to 1. Changing a formula and letting new numbers mingle with old ones in the same chart is the silent way to destroy the longitudinal record; the version column makes formula changes visible and chart annotations possible.
- Wrinkle/pore ranges re-fitted to the 512 px scale on the five evenly-lit reference faces (`WRINKLE 0.10–0.50 → 0.25–0.75`, `PORE 0.01–0.15 → 0.03–0.22`); pigmentation / erythema / uniformity distributions were unchanged and keep their v1 ranges.
- **Wrinkle range re-fitted a second time, now against ground truth** (Session 6): FFHQ-Wrinkle (n=1000 hand-annotated faces, ROI-restricted, CLAHE-matched to production) measures real-face `wrinkle_raw` p5–p95 = [0.197, 0.619] — the reference-face ceiling of 0.75 was never reached, so the top quarter of the 0–10 wrinkle scale was dead range. `WRINKLE_RAW_RANGE` is now (0.20, 0.62), pinned to the measured distribution by `tests/test_validation_benchmarks.py` (`uv run pytest -m validation`).

7. What I cut for time

- **Per-cohort percentile scoring.** Discussed above. Needs a real cohort.
- **Identity verification on intake** (face-embedding compare against the patient's first-visit embedding to catch wrong-patient uploads). Designed, not implemented.
- **Promoting `LIVE_CAPTURE_STABILITY_FRAMES` from JS hard-code into a Python config injected via component args.** Roughly 30 lines; the current single-source-of-truth lie in §6 above will go away when this lands.
- **Latency under real-network LLM call.** §4's number excludes the Anthropic / Gemini round-trip. I have not benchmarked the full user-perceived latency from photo-upload to explanation rendered.
- **Mobile / iPad viewport.** The custom component is sized for laptop (1200 px iframe). Touch gestures untested.
- **Real customer development.** Per the AI Fund operating model observed at the Beauty Clinic OS breakfast, ideation + market validation are owned by the fund's internal team. EIRs are filtered on technical execution. I respected that boundary.

8. What I'd do next (in priority order)

1. **Sit down with real practitioners and watch them work.** 3–5 receptionists and aesthetic-clinic doctors, observed mid-intake in their actual rooms with their actual patients. The pricing model, the feature roadmap, and the question of whether the longitudinal chart even ends up in the patient conversation are all things those sessions answer better than any thought experiment I run alone. This is the highest-leverage single thing I can do before writing more code.
2. **Collect real intake data and partner with a clinic to train a true end-to-end model.** The current deterministic CV pipeline is a defensible Phase-1 wedge precisely because it works without any data. Phase 2 is using the longitudinal photos a deployment generates — under proper consent — to train an actual end-to-end skin-progression model. That only becomes possible once a clinic is onboard and the consent pipeline is in place; the deterministic layer in §1 is what lets us reach that point honestly, by being useful from day one without needing the data we don't yet have.
3. **Identity verification on intake** (cut from time, design ready).